

PROGRAM 5: Write a program to implement stack using arrays.

THEORY:

WRITE ABOUT STACK AND ITS OPERATIONS

ALGORITHM: Stack Using Arrays

Step 1: Initialization

- Define a constant MAX for maximum size of stack.
- Create an array stack[MAX].
- Initialize top = -1.

Step 2: PUSH (Insert element)

1. Check if top == MAX-1.
 - If **true** → *Stack Overflow*.
 - Else:
 - Increment top = top + 1.
 - Set stack[top] = item.

Step 3: POP (Delete element)

1. Check if top == -1.
 - If **true** → *Stack Underflow*.
 - Else:
 - Store stack[top] in item.
 - Decrement top = top - 1.
 - Return item.

Step 4: PEEK (View top element)

1. If top == -1 → *Stack is empty*.
2. Else → return stack[top].

Step 5: DISPLAY

1. If top == -1 → print *Stack is empty*.
2. Else → print elements from top to 0.

PROGRAM:

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
// PUSH operation
```

```
void push(int item) {
```

```
    if (top == MAX - 1) {
```

```
        printf("Stack Overflow!\n");
```

```
    } else {
```

```
        top++;
```

```
        stack[top] = item;
```

```
        printf("%d pushed to stack\n", item);
```

```
    }
```

```
}
```

```
// POP operation
```

```
void pop() {
```

```
    if (top == -1) {
```

```
        printf("Stack Underflow!\n");
```

```
    } else {
```

```
        printf("%d popped from stack\n", stack[top]);
```

```
        top--;
```

```
    }
```

```
}
```

```
// PEEK operation
```

```
void peek() {  
    if (top == -1) {  
        printf("Stack is empty!\n");  
    } else {  
        printf("Top element: %d\n", stack[top]);  
    }  
}
```

```
// DISPLAY operation
```

```
void display() {  
    if (top == -1) {  
        printf("Stack is empty!\n");  
    } else {  
        printf("Stack elements: ");  
        for (int i = top; i >= 0; i--) {  
            printf("%d ", stack[i]);  
        }  
        printf("\n");  
    }  
}
```

```
int main() {  
    push(10);  
    push(20);  
    push(30);  
    display();  
    peek();  
    pop();  
    display();  
}
```

```
    return 0;  
}
```

OUTPUT:

10 pushed to stack

20 pushed to stack

30 pushed to stack

Stack elements: 30 20 10

Top element: 30

30 popped from stack

Stack elements: 20 10